Open in app ↗ Sign up Sign in

Medium Search Write

Aprendizaje por refuerzo (RL) — Capítulo 3: Multi-armed bandit— Parte 1: Epsilon-greedy y epsilon-decreasing



Joan Cerretani · Follow

12 min read · Dec 6, 2023



Foto de Sigmund en Unsplash

Bienvenidos al tercer capítulo de esta serie sobre aprendizaje por refuerzo.

En este capítulo vamos a estudiar el problema del bandido de múltiples brazos, o mejor conocido como Multi-armed bandit. Aquí nos vamos a enfrentar al primer problema que vamos a resolver con aprendizaje por refuerzo y para esto vamos a utilizar varías técnicas.

Este capítulo va a ser esencial para entender el resto de problemas que vamos a ver a lo largo de esta serie, así que acompáñame en este viaje.

Arranquemos!

A/B testing

La implementación de un modelo es un proceso que requiere dedicación y recursos considerables en el ámbito de la ciencia de datos. Un ejemplo concreto implica diseñar un modelo que ofrezca recomendaciones de productos en línea a compradores, basándose en su historial de compras y preferencias. Este modelo se desarrolla inicialmente en un entorno controlado, utilizando datos históricos de clientes como punto de partida. Sin embargo, su verdadero valor se materializa cuando se despliega para que los usuarios finales puedan interactuar con él.

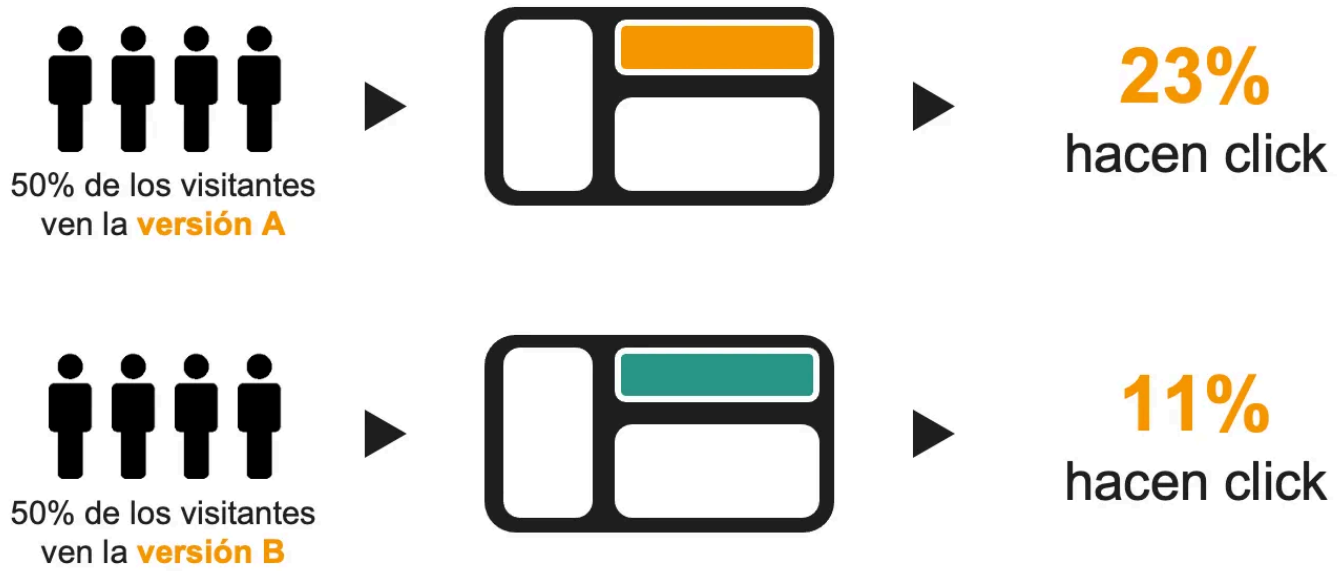


En esta fase de despliegue, es esencial evaluar la eficiencia y el rendimiento del modelo en un grupo de usuarios de prueba cuidadosamente seleccionado. Este grupo se considera representativo de la población en general. Esta evaluación nos proporciona información crítica sobre el desempeño del modelo en condiciones del mundo real. En caso de que el modelo no funcione como se esperaba, esta etapa permite identificar problemas y realizar ajustes antes de que afecten a un público más amplio.

En situaciones como estas, es común utilizar una metodología experimental conocida como prueba A/B o **A/B testing**. Un experimento A/B es una evaluación aleatoria que implica dos variantes (denominadas A y B) con el propósito de determinar cuál de ellas es más efectiva, utilizando una métrica predefinida como criterio de éxito. En nuestro ejemplo anterior, la métrica

podría ser la cantidad de compras realizadas por un usuario en nuestra tienda en línea.

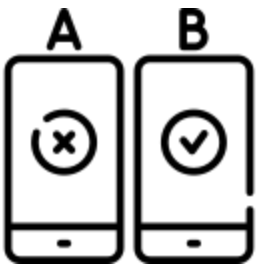
Para ilustrar este concepto, consideremos el siguiente escenario: tenemos dos versiones de nuestra página web, una con el botón de compra en color naranja y la otra en color azul. Queremos determinar cuál de estas opciones incita a más usuarios a hacer clic en el botón de compra. Para hacerlo, mostramos la variante A a la mitad de nuestros usuarios y la variante B a la otra mitad. Después de un período de tiempo suficiente para recopilar datos significativos, aplicamos análisis estadísticos para determinar si una de las variantes supera significativamente a la otra en términos de la tasa de clics. Este enfoque nos permite tomar decisiones basadas en datos sólidos y ajustar nuestro modelo o diseño en consecuencia.



Ejemplo de grupos de prueba para definir diseño de página web a partir de un test A/B.

Los experimentos A/B son ampliamente empleados en el diseño de páginas web y aplicaciones, y su utilidad se extiende a diversos ámbitos. Algunos ejemplos de aplicación incluyen la determinación de precios para productos, estrategias de campañas políticas o evaluación del rendimiento de APIs.

Los experimentos A/B ejercen un profundo impacto en las organizaciones. Un caso ilustrativo se dio en el año 2009, cuando Google realizó pruebas con 41 tonalidades de azul distintas para los anuncios de Gmail y los enlaces de resultados de búsqueda. Este aparentemente pequeño ajuste se tradujo en un incremento adicional de **$200 millones anuales en ingresos** por publicidad para la empresa.



El enfoque A/B nos otorga una herramienta valiosa para adquirir confianza cuantitativa en torno a la viabilidad de una hipótesis. Esta metodología implica un esfuerzo de desarrollo relativamente bajo y nos permite cambiar de dirección con rapidez, un elemento esencial para alcanzar resultados satisfactorios.

No obstante, los experimentos A/B presentan una limitación significativa. Exigen acumular una cantidad considerable de datos con significación estadística antes de que podamos tomar una decisión entre la variante A y la B, y esto implica detener manualmente el experimento cuando consideremos que tenemos una muestra suficiente. Una alternativa podría ser implementar un mecanismo adaptativo que evalúe de manera dinámica las alternativas y converja hacia la mejor solución de manera más eficiente que esperar a acumular un gran número de muestras para llevar a cabo un experimento A/B.

Para abordar este desafío, podemos modelar el problema como un problema de “Multi-armed bandit” (MABP, por sus siglas en inglés). Este enfoque permite una toma de decisiones más ágil y eficaz en entornos en los que se busca optimizar resultados de manera continua.

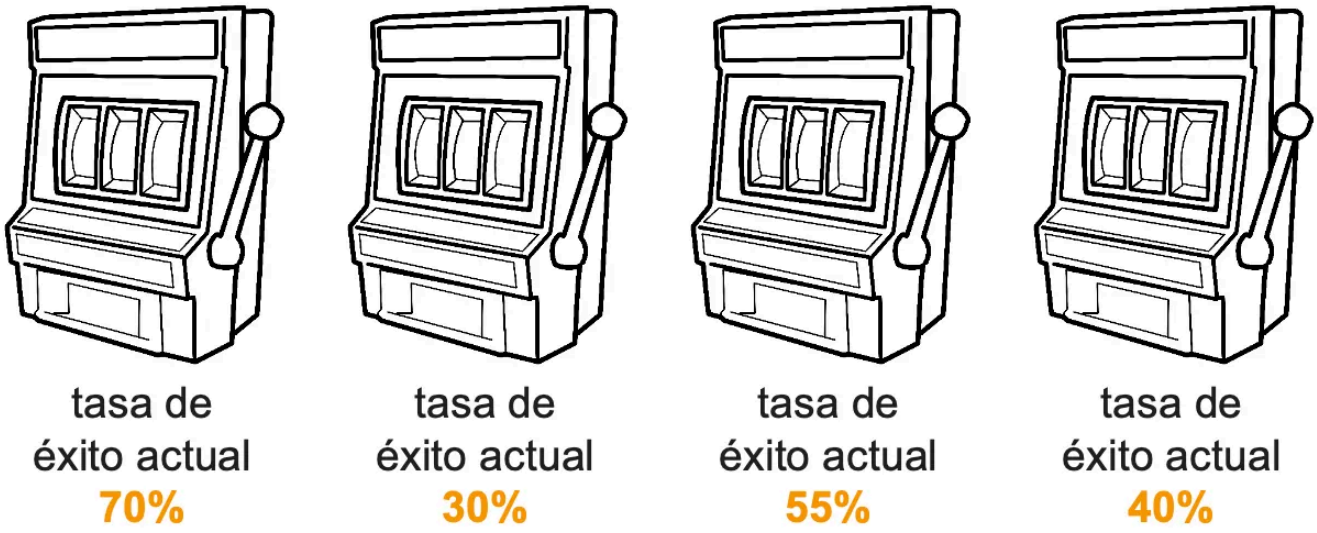
Multi-armed bandit

Los algoritmos de “Multi-armed bandit” emplean el aprendizaje adaptativo para evaluar diversas alternativas, que en este contexto pueden superar las dos opciones convencionales, con el objetivo de identificar la mejor entre ellas. En esta ocasión, exploraremos nuestro primer algoritmo de aprendizaje por refuerzo.

Para ilustrar este concepto, consideremos un ejemplo clásico relacionado con el bandido de múltiples brazos: imaginemos a un jugador en un casino que se encuentra ante la encrucijada de decidir en cuál de las múltiples máquinas tragamonedas apostar. Cada una de estas tragamonedas posee una tasa de éxito desconocida para el jugador. En su búsqueda por maximizar sus ganancias, el jugador decide probar cada una de las tragamonedas y registra cuidadosamente el rendimiento que obtiene de cada una de ellas.

Después de un período de juego, el jugador comienza a notar que una de las máquinas le ha proporcionado más ganancias que las demás. Sin embargo, aún alberga ciertas dudas y no está completamente seguro de que esta sea la máquina que le brindará el mayor rendimiento a largo plazo. Esto plantea una pregunta fundamental: **¿Cómo y cuándo puede el jugador adquirir la certeza de haber identificado la máquina más rentable entre todas las disponibles?**

Para responder a esta interrogante, surgen diversas estrategias y enfoques. Los algoritmos desarrollados con el propósito de encontrar un orden óptimo en el cual seleccionar las tragamonedas se conocen como “algoritmos de bandido de múltiples brazos” o “bandit algorithms”. Estos algoritmos se convierten en herramientas esenciales en la toma de decisiones en situaciones donde la incertidumbre y la maximización de recompensas son componentes críticos, y se aplican en campos que van más allá de las tragamonedas, como la optimización de anuncios en línea, la administración de recursos en empresas, y la toma de decisiones médicas, entre otros.



Tragamonedas con diferentes tasas de éxito esperadas según las pruebas realizadas.

Exploración y explotación

No obstante, es crucial abordar dos conceptos fundamentales para comprender tanto el funcionamiento de los algoritmos de selección de acciones, como cualquier otro algoritmo perteneciente al campo del aprendizaje por refuerzo. Estos conceptos son la **exploración** y la **explotación**.

La **exploración** se refiere al proceso mediante el cual un agente o jugador prueba diversas opciones, incluso aquellas que no parecen generar un retorno inmediato y sustancial, con el objetivo de determinar si en algún momento estas opciones pueden ser más beneficiosas. Durante la fase de exploración, el jugador no busca maximizar sus recompensas de inmediato; en su lugar, busca adquirir información sobre el rendimiento de cada opción disponible.

Por otro lado, la **explotación** ocurre cuando el jugador decide enfocarse en una opción particular que ha demostrado ser más rentable hasta el momento, con la intención de comenzar a generar ganancias consistentes a partir de esa elección.

A diferencia de un enfoque tradicional de prueba A/B, donde se divide claramente el proceso en fases de exploración y explotación, comenzando con una fase de pruebas para explorar todas las opciones disponibles y luego, después de un período de tiempo predeterminado, seleccionando la opción que parece ser la más prometedora, los algoritmos de selección de acciones tipo “bandit” alternan entre estos dos estados de manera adaptativa en función del rendimiento de cada acción.

El objetivo de los algoritmos de selección de acciones tipo “bandit”, similar al enfoque de prueba A/B, es identificar la mejor opción dentro de un conjunto de alternativas y, posteriormente, explotarla para maximizar las ganancias. Sin embargo, para lograr este equilibrio, el jugador debe dedicar parte de su tiempo a explorar diferentes opciones y, al mismo tiempo, explotar intermitentemente la mejor opción para minimizar las pérdidas potenciales al elegir acciones subóptimas. Este equilibrio entre exploración y explotación es una característica fundamental del Problema del Bandido Multi-Armed.

Entonces, surge la pregunta: ¿Cómo decide el jugador cuándo explorar y cuándo explotar? La respuesta a esta cuestión depende del tipo de algoritmo que se esté utilizando para abordar el Problema del Bandido Multi-Armed, ya que existen diversas variaciones de estos algoritmos, cada una con su propio enfoque y estrategia de toma de decisiones.

Estrategias epsilon-greedy y epsilon-decreasing

Existen estretegias para contestar dicha pregunta. Dos de ellas son las conocidas como epsilon-greedy y epsilon-decreasing. De hecho, estas estrategias no sólo son útiles para resolver este problema en particular, sino otros problemas de aprendizaje por refuerzo, como veremos más adelante.

La estrategia epsilon-greedy es una técnica fundamental en la resolución del problema del Multi-Armed Bandit Problem. Su principal objetivo es equilibrar la exploración y la explotación de las acciones disponibles. Pero, antes de entrar en detalles sobre cómo funciona esta estrategia, es esencial entender el concepto de epsilon (ϵ).

En este contexto, ϵ es una constante que varía entre *0* y *1*, y su valor determina la proporción de tiempo que el agente dedicará a la exploración en comparación con la explotación. Por ejemplo, si $\epsilon = 0.1$, el agente gastará el *10%* de su tiempo explorando nuevas acciones y el *90%* restante explotando la acción que ha demostrado ser más rentable hasta ese momento.

Para implementar la estrategia epsilon-greedy, primero se selecciona un valor específico para ϵ. En nuestro ejemplo, utilizaremos $\epsilon = 0.1$. Durante la ejecución del algoritmo, se generan números aleatorios en el rango de *0* a *1* en cada iteración. Si el valor aleatorio es menor que ϵ, el agente elige una acción al azar para explorar. Por otro lado, si el valor aleatorio es mayor que ϵ, el agente selecciona la acción que ha demostrado tener el mayor retorno hasta ese momento para explotarla.

Esta estrategia brinda flexibilidad, ya que permite personalizar el grado de exploración que se desea en función de la tolerancia al riesgo de tomar decisiones subóptimas.

Una extensión interesante de la estrategia epsilon-greedy es la técnica denominada "epsilon-decreasing". En esta versión, a diferencia de la estrategia original, ϵ no permanece constante durante toda la ejecución, sino que disminuye en cada iteración. Como su nombre indica, ϵ decrece gradualmente.

Función de valor

Lo último que nos queda saber para poder realizar la implementación de este algoritmo es cómo definir una noción de valor para que el agente pueda elegir la acción que le genere mayor retorno. Para lograr esto generamos la función de valor $Q(s,a)$ que representa la recompensa esperada al tomar una acción a en un estado s.

En nuestro caso el estado s es siempre el mismo, ya que siempre estamos ante la misma situación de tener que elegir una opción entre todas las

disponibles. Es decir, el estado del juego no cambia, solo lo hace nuestro conocimiento sobre el entorno. Se podría considerar que este es un entorno episódico, de un solo episodio. Entonces, en nuestro caso podemos definir nuestra función de valor *Q* como:

$$Q_k(a) = \frac{1}{k}\left(r_1 + r_2 + ... + r_k\right)$$

Donde *k* es la cantidad de veces que se tomó la acción *a* y r_i es la recompensa obtenida en cada episodio como resultado de elegir dicha acción. Esto no es más que el promedio de recompensas obtenidas para dicha acción, por lo que para un valor de *k* muy alto $Q_k(a)$ converge al retorno medio esperado de dicha acción.

Esta ecuación nos obliga a guardar el histórico de todas las recompensas obtenida para cada acción, y eso no parece ser eficiente en términos de memoria. Con un poco de manipulación aritmética podemos modificar esta ecuación y reescribirla como:

$$\begin{aligned}
Q_{k+1}(a) &= \frac{1}{k+1}\sum_{i=1}^{k+1} r_i \\
&= \frac{1}{k+1}\left(r_{k+1} + \sum_{i=1}^{k} r_i\right) \\
&= \frac{1}{k+1}\left(r_{k+1} + kQ_k(a) + Q_k(a) - Q_k(a)\right) \\
&= \frac{1}{k+1}\left(r_{k+1} + (k+1)Q_k(a) - Q_k(a)\right) \\
&= Q_k(a) + \frac{1}{k+1}\left[r_{k+1} - Q_k(a)\right]
\end{aligned}$$

De esta forma solo tenemos que hacer seguimiento de las veces que se seleccionó cada acción (*k*) y actualizar la función de valor a partir de la

ecuación anterior.

De este modo, en cada iteración se generará un número aleatorio y se lo comparará con el valor actual de ϵ. Si dicho número es menor que ϵ, entonces se lleva a cabo una acción aleatoria. Si no, se opta por la acción a^* que maximice a nuestra función Q:

$$a^* = argmax_a \left(Q(a)\right)$$

Ejemplo

Ahora sí estamos en condiciones de volver al problema de las tragamonedas.

Supongamos que tenemos cuatro máquinas tragamonedas, donde las probabilidades de ganar en cada una son *0.25*, *0.35*, *0.55* y *0.60*, respectivamente. Esta probabilidad es a priori desconocida para nosotros, y el objetivo es averiguar cuál de ellas resulta más rentable.

Como primer paso vamos a generar nuestro vector de recompensas o retornos esperados, al que llamaremos Q, y que al inicio es desconocido para nosotros. Por ende, podemos inicializarlo con un *0* para todas las acciones (cada componente corresponde a una acción). Matemáticamente, esto implica que al comienzo tendremos que *Q=[0, 0, 0, 0]*.

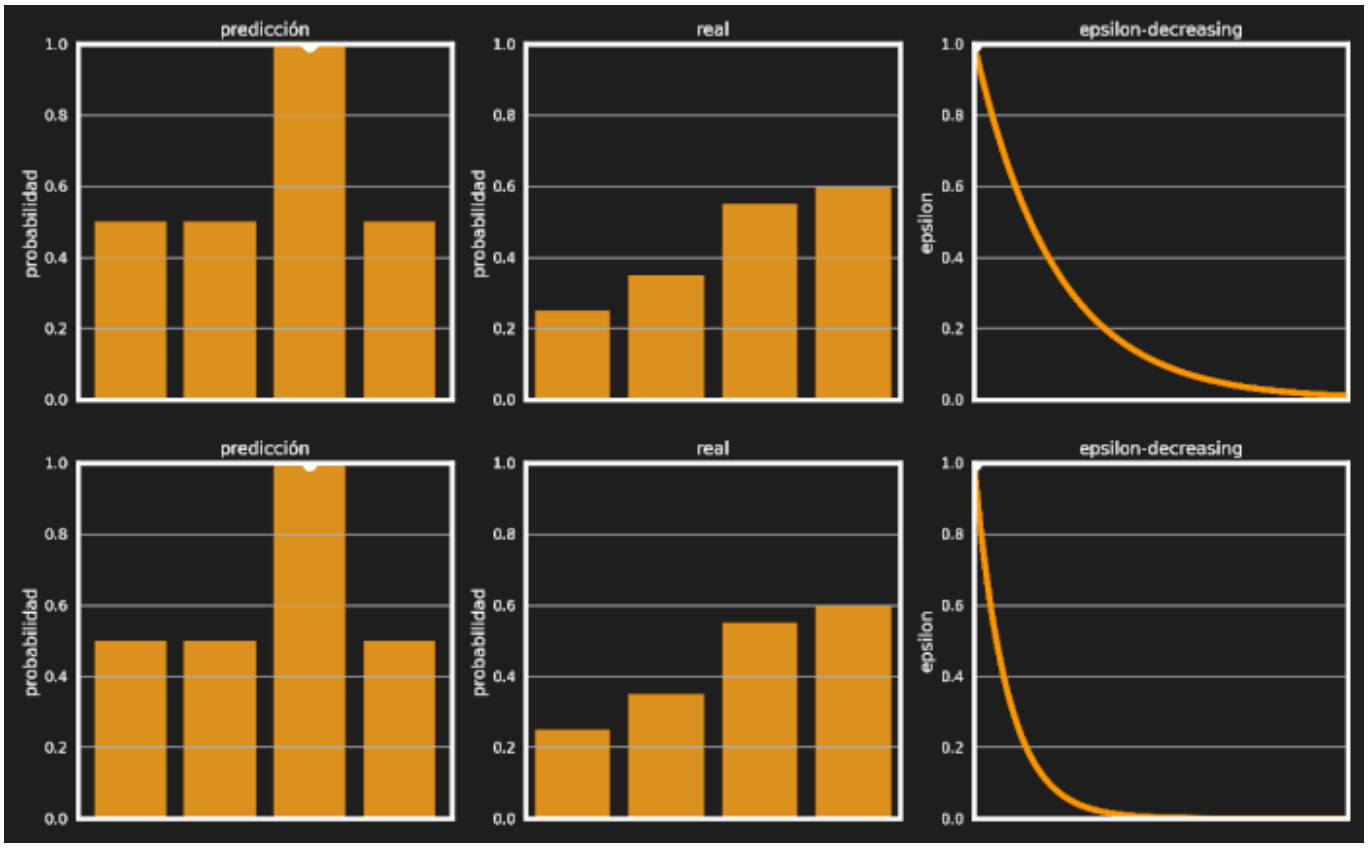
Por otro lado, vamos a definir nuestro ϵ inicial en *1.0*. Este valor va a disminuir exponencialmente en cada iteración convergiendo a *0*.

A continuación comenzamos a iterar. En cada paso se elige una acción y, como al inicio el valor de ϵ es muy cercano a *1*, es muy posible que elijamos acciones aleatorias. Digamos que nuestro agente elige tomar la primera acción (en nuestro caso seria jugar en la primer tragamonedas) y luego de hacer esta acción obtiene una recompensa de *1*, es decir, la tragamonedas resultó ganadora. Posteriormente actualizamos nuestro vector de recompensas Q a partir de la ecuación de $Q_{k+1}(a)$:

$$Q = \left[0 + \frac{1}{0+1}(1-0),\ 0,\ 0,\ 0\right] = [1,\ 0,\ 0,\ 0]$$

Este paso se repite de forma iterativa N veces, disminuyendo el valor de ϵ en cada iteración, hasta finalmente converger.

A continuación puede ver una animación del entrenamiento. En la columna izquierda se encuentra la estimación del valor de la acción y el punto blanco indica la acción seleccionada en cada paso. La segunda columna muestra las probabilidades reales de cada “tragamoneda” y a la derecha el valor de ϵ. La primera y segunda fila son dos simulaciones para diferentes decaimientos de ϵ.



Es importante tener en cuenta que nuestro agente en el proceso de toma de decisiones pasa por dos fases distintas: exploración y explotación. Esto significa que nuestro vector *Q*, que representa el valor esperado de las acciones en un estado dado, no convergerá necesariamente al valor real de las recompensas a largo plazo. Esto se debe a que, con el tiempo, el agente tiende a enfocarse en la explotación de la acción que parece generar el mayor retorno inmediato.

Como resultado de esta dinámica, nuestras estimaciones de valor para las acciones que han demostrado ser más prometedoras tendrán menos incertidumbre, ya que han sido exploradas en mayor medida. Sin embargo, las acciones menos rentables recibirán menos atención en la fase de exploración a medida que el agente se inclina hacia la explotación, lo que las alejará de su valor real.

Es fundamental comprender que este fenómeno no es una preocupación significativa para nosotros en el contexto del aprendizaje por refuerzo. Nuestro objetivo principal no es obtener una estimación precisa de los retornos de cada acción individual, sino identificar cuál de ellas maximiza nuestro retorno global. En otras palabras, estamos más interesados en determinar la acción óptima que nos permitirá obtener la mayor recompensa acumulada a lo largo del tiempo. Esta distinción nos permite centrarnos en la toma de decisiones estratégicas en lugar de una precisión absoluta en las estimaciones de valor.

Implementación

En resumen, el bandit algorithm con epsilon-decreasing se puede implementar a partir de los siguientes pasos:

Algorithm 1 Bandit algorithm with epsilon-decreasing

1: Initializa action-value vector Q
2: Initializa $\epsilon = 1$
3: **for** episode$= 1, N$ **do**
4: With probability ϵ select a random action a
5: otherwise select $a = argmax_a\,(Q(a))$
6: Execute action a in environment and observe reward r
7: Update the vector Q from the equation:

$$Q_{k+1}(a) = Q_k(a) + \frac{1}{k+1}\,[r - Q_k(a)]$$

8: Decreased the value of ϵ
9: **end for**

En la implementación tenga en cuenta que se debe llevar el conteo del valor k para cada acción independientemente, es decir que k también tiene dependencia con la acción al igual que Q.

En la última parte de esta capítulo vamos a ver como implementar este algoritmo (y el resto de los algoritmos de este capítulo) en Python. Mientras tanto puede ir revisando la notebook con la implementación del modelo en el repositorio de GitHub:

GitHub - jcerretani/AprendizajePorRefuerzo

Contribute to jcerretani/AprendizajePorRefuerzo development by creating an account on GitHub.

github.com

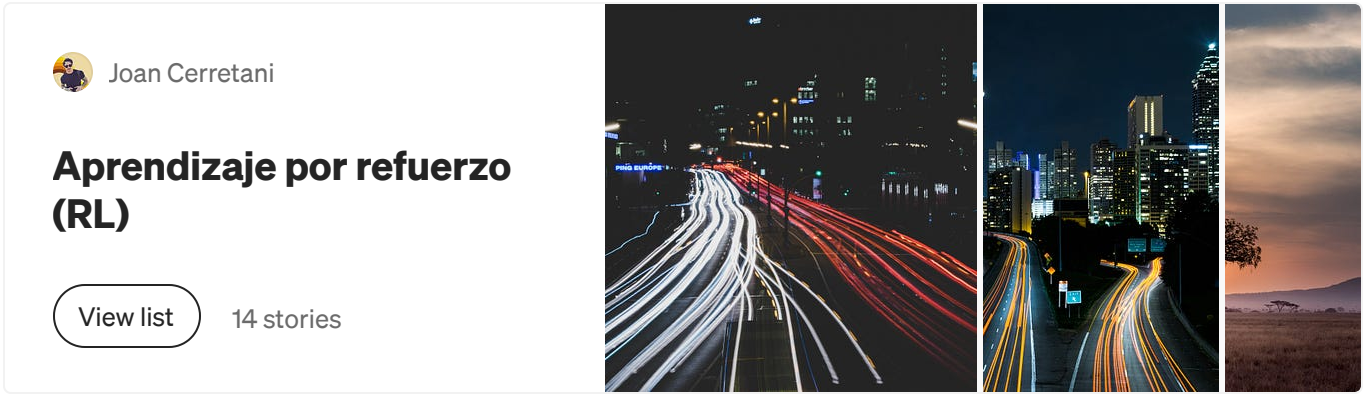
Y hasta aquí la primera parte, en la segunda parte vamos a ver otras estrategias como son Upper Confidence Bound, Softmax y Multi-armed bandit con muestreo de Thompson, así que no te la pierdas!.

Lecturas sugeridas

- Ve a la **siguiente parte** de este capítulo: Aprendizaje por refuerzo (RL) — Capítulo 3: Multi-armed bandit — Parte 2: UCB, Softmax y muestreo de Thompson

- Ve al **capítulo anterior:** Aprendizaje por refuerzo (RL) — Capítulo 2: Introducción — Parte 1: ¿Qué es el aprendizaje por refuerzo?

Y también puedes visitar el **resto de artículos** sobre Aprendizaje por refuerzo:



Referencias

- https://medium.com/opex-analytics/multi-armed-bandits-101-6f4ac62b6bd6
- https://towardsdatascience.com/solving-the-multi-armed-bandit-problem-b72de40db97c
- https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf

Reinforcement Learning | Artificial Intelligence | Multi Armed Bandit | Aprendizaje Por Refuerzo | Inteligencia Artificial



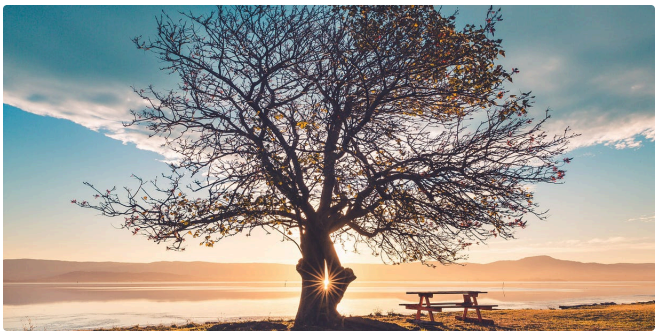
Written by Joan Cerretani

Follow

17 Followers

Soy Joan Cerretani, Lic. en Ciencias Físicas de la UBA y Mtr. en Ciencia de Datos.

More from Joan Cerretani



Joan Cerretani

Aprendizaje por refuerzo (RL)— Capítulo 4: Min-Max y Alpha-Beta...

En la parte anterior explicamos el algoritmo Min-Max como una estrategia de simular...

9 min read · Dec 7, 2023

1



Joan Cerretani

Aprendizaje por refuerzo (RL)— Capítulo 2: Introducción—Parte 3...

Bienvenidos a la tercer y última parte de este capítulo donde nos vamos a centrar en las...

11 min read · Dec 5, 2023



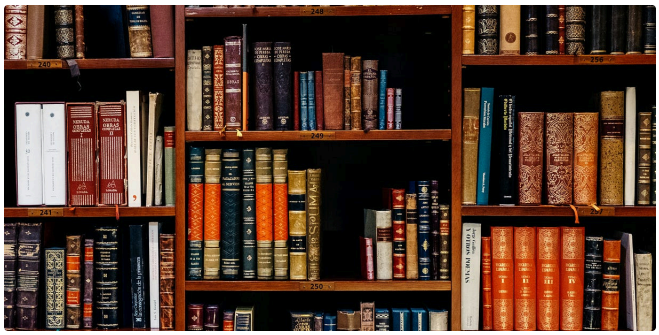
Joan Cerretani

Aprendizaje por refuerzo (RL)— Capítulo 1: Historia del aprendizaj...

Bienvenidos al primer capítulo dedicado al tema del aprendizaje por refuerzo o también...

10 min read · Dec 4, 2023

11



Joan Cerretani

Aprendizaje por refuerzo (RL)— Capítulo 1: Historia del aprendizaj...

Bienvenidos a la segunda parte de la historia del aprendizaje por refuerzo.

16 min read · Dec 4, 2023

16

See all from Joan Cerretani

Recommended from Medium





Henry Wu

Intro to Reinforcement Learning: Monte Carlo to Policy Gradient

This post is an intro to reinforcement learning, in particular, Monte Carlo methods, Tempora...

16 min read · Feb 14, 2024

188

Matthias Dittrich in UX Planet

Stop designing chat-based AI tools.

It is time to evolve AI tools beyond prompt-based interfaces and consider new mental...

5 min read · Apr 21, 2024

959 20

Lists



AI Regulation

6 stories · 434 saves



ChatGPT

21 stories · 616 saves



Generative AI Recommended Reading

52 stories · 1000 saves



ChatGPT prompts

47 stories · 1515 saves



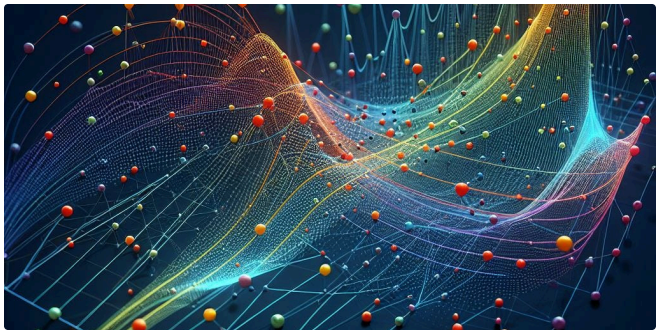
Ameer Omidvar in Bootcamp

Apple's all new design language

My name is Ameer, currently the designer of Sigma. I've been in love with design since i...

5 min read · Feb 14, 2024

5.3K 56



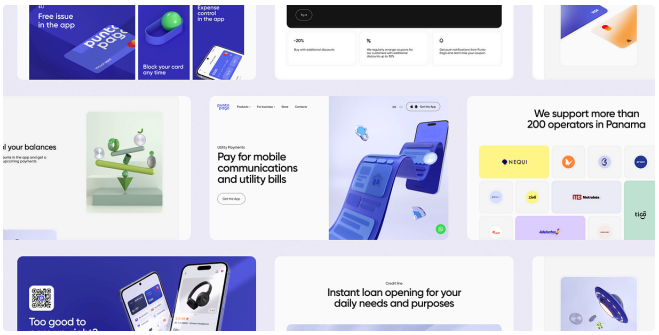
Tim Sumner in Towards Data Science

A New Coefficient of Correlation

What if you were told there exists a new way to measure the relationship between two...

10 min read · Mar 31, 2024

3.4K 43



dev.family

UX/UI-trends in 2024

The 2023 year was rich in technological innovations: we witnessed the introduction o...

13 min read · Feb 28, 2024

1.8K 12



Mohamed Yosef

Reinforcement Learning — Policy gradient 101

There are two approaches for solving any RL problem; value-based methods and policy-...

5 min read · Feb 19, 2024

110

See more recommendations